# Práctica ASO: compilación de un kernel Linux propio

## 1. Objetivo

En esta práctica he compilado un kernel Linux propio a partir del que estaba disponible en `trixie-backports`. El kernel final es **7.2.6-sergio-aso**.

La idea no era simplemente compilar el kernel completo con la configuración de Debian, sino ajustarlo al hardware real de mi portátil. Para ello partí de la configuración del kernel que ya usaba mi equipo, recogí los módulos cargados y apliqué `localmodconfig`. Después revisé manualmente los controladores importantes antes de generar los paquetes `.deb`.

También dejé probado el arranque normal desde GRUB y el arranque directo desde UEFI sin pasar por GRUB. Como mi equipo tenía Secure Boot activado, lo desactivé desde la UEFI para poder arrancar un kernel compildo por mí sin firmarlo.

> **Nota de seguridad:** durante todo el proceso mantuve instalados los kernels oficiales de Debian. De esta forma, si el kernel compilado no arrancaba, podía volver a arrancar desde GRUB con el kernel anterior.

---

## 2. Equipo y comprobaciones previas

Antes de empezar comprobé que había espacio suficiente. La compilacion se hizo dentro de `~/practica-kernel`, que está en `/home`, donde tenía mucho espacio libre. También comprobé `/boot`, porque al instalar el kernel se generan ahí el kernel y el initramfs.

Mi equipo usa:

- CPU/plataforma Intel Meteor Lake.
- Gráfica Intel integrada con el driver `i915`.
- Wi-Fi Realtek RTL8852BE con `rtw89_8852be`.
- SSD NVMe, controlado a través de Intel VMD.
- Audio Intel/Realtek.
- USB xHCI, cámara UVC y Bluetooth Realtek.
- Arranque UEFI y raíz en LVM.

Comprobación del espacio disponible:

```
sergio@Sergio-PC:~$ df -h /
df -h /home
df -h /tmp
S.ficheros                     Tamaño Usados  Disp Uso% Montado en
/dev/mapper/vg_debian-lv_root    46G   7,4G   36G  18% /
S.ficheros                     Tamaño Usados  Disp Uso% Montado en
/dev/mapper/vg_debian-lv_home   261G    28G  219G  12% /home
S.ficheros                    Tamaño Usados  Disp Uso% Montado en
/dev/mapper/vg_debian-lv_tmp   7,3G    41M  6,9G   1% /tmp

sergio@Sergio-PC:~/practica-kernel$ df -h /boot
S.ficheros     Tamaño Usados  Disp Uso% Montado en
/dev/nvme0n1p2   921M   184M  674M  22% /boot
```

---

## 3. Preparación del sistema

Primero habilité el repositorio `trixie-backports`, porque el paquete del código fuente del kernel no estaba disponible en los repositorios normales.

Archivo creado:

```
/etc/apt/sources.list.d/trixie-backports.list
```

Contenido:

```
deb http://deb.debian.org/debian trixie-backports main contrib non-free non-free-firmware
```

Después actualicé APT e instalé las herramientas necesarias junto con el código fuente del kernel:

```
sergio@Sergio-PC:~/practica-kernel$ sudo apt update
sergio@Sergio-PC:~/practica-kernel$ sudo apt install build-essential bc bison flex libncurses-dev libssl-dev libelf-dev dwarves fakeroot rsync git linux-source-7.2/trixie-backports
```

El paquete que instalé fue `linux-source-7.2`, versión `7.2.6-1~bpo13+1`:

```
sergio@Sergio-PC:~/practica-kernel$ apt policy linux-source-7.2
linux-source-7.2:
  Instalados: 7.2.6-1~bpo13+1
  Candidato:  7.2.6-1~bpo13+1
  Tabla de versión:
 *** 7.2.6-1~bpo13+1 100
        100 http://deb.debian.org/debian trixie-backports/main amd64 Packages
        100 /var/lib/dpkg/status
```

Comprobé además el estado de Secure Boot y el kernel de partida:

```
sergio@Sergio-PC:~/practica-kernel$ mokutil --sb-state
SecureBoot enabled
sergio@Sergio-PC:~/practica-kernel$ uname -r
6.12.111+deb13-amd64
```

---

## 4. Preparar la configuración

Creé el directorio de trabajo en mi home, extraje el código fuente y copié la configuración del kernel que ya arrancaba correctamente en mi portátil. Esto es importante porque así se parte de una base conocida que soporta el hardware real del equipo.

```
sergio@Sergio-PC:~$ mkdir -p ~/practica-kernel
sergio@Sergio-PC:~$ cd ~/practica-kernel
sergio@Sergio-PC:~/practica-kernel$ tar -xaf /usr/src/linux-source-7.2.tar.xz
sergio@Sergio-PC:~/practica-kernel$ cd linux-source-7.2/
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ cp /boot/config-$(uname -r)
.config
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ make olddefconfig
#
# configuration written to .config
#
```

Antes de reducir módulos revisé qué hardware y qué módulos cargaba el equipo con estos comandos:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ lspci -k
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ lsusb
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ lsmod
```

Parte de la salida relevante de `lspci -k`:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ lspci -k
00:02.0 VGA compatible controller: Intel Corporation Meteor Lake-P [Intel
Graphics] (rev 08)
    Kernel driver in use: i915
    Kernel modules: i915, xe
...
00:0e.0 RAID bus controller: Intel Corporation Volume Management Device NVMe
RAID Controller Intel Corporation
    Kernel driver in use: vmd
    Kernel modules: vmd
...
00:1f.3 Audio device: Intel Corporation Meteor Lake-P HD Audio Controller (rev
20)
    Kernel driver in use: snd_hda_intel
    Kernel modules: snd_hda_intel, snd_sof_pci_intel_mtl
...
01:00.0 Network controller: Realtek Semiconductor Co., Ltd. RTL8852BE PCIe
802.11ax Wireless Network Controller
    Kernel driver in use: rtw89_8852be
    Kernel modules: rtw89_8852be
02:00.0 Non-Volatile memory controller: Shenzhen Techwinsemi Technology Co.,
Ltd. PCIe Gen4 x4 M.2 2280 (rev 01)
    Kernel driver in use: nvme
    Kernel modules: nvme
```

También se detectaron cámara, lector de tarjetas y Bluetooth:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ lsusb
Bus 003 Device 002: ID 25a7:fa61 Areson Technology Corp Elecom Co., Ltd MR-K013
Multicard Reader
Bus 003 Device 003: ID 1bcf:1249 Sunplus Innovation Technology Inc. PC Camera
Bus 003 Device 004: ID 0bda:0129 Realtek Semiconductor Corp. RTS5129 Card Reader
Controller
Bus 003 Device 005: ID 0bda:b85b Realtek Semiconductor Corp. Bluetooth Radio
```

---

## 5. Quitar módulos innecesarios

Esta es la parte principal de la práctica. Con el kernel de Debian arrancado normalmente, guarde los módulos realmente cargados por el equipo:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ lsmod > ~/practica-
kernel/modulos-usados.txt
```

A continuación ejecuté `localmodconfig`. Esta opción usa ese listado para desactivar en la configuración muchos módulos que no estaban en uso en mi máquina:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ yes '' | make
LSMOD=~/practica-kernel/modulos-usados.txt localmodconfig
using config: '.config'
crct10dif_pclmul config not found!
crc32c_generic config not found!
crc32c_intel config not found!
...
WARNING: CRYPTO_SIMD is required, but nothing in the
  current config selects it.
module ip_tables did not have configs CONFIG_IP_NF_IPTABLES_LEGACY
module crypto_simd did not have configs CONFIG_CRYPTO_SIMD
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ make olddefconfig
#
# configuration written to .config
#
```

Los avisos anteriores no pararon la configuración. Algunos nombres de módulos no coinciden directamente con el nombre de su opción en `Kconfig`; por eso después revisé manualmente lo que no podía faltar para arrancar y usar el portátil.

## Módulos revisados y mantenidos

No desactivé de forma ciega controladores de arranque. Comprobé que la configuración final incluyera:

- `BLK_DEV_NVME` y `NVME_CORE` para el SSD NVMe.
- `VMD` porque el portátil usa Intel VMD para acceder al SSD.
- `DRM_I915` para la gráfica Intel.
- `RTW89_8852BE` para la Wi-Fi Realtek RTL8852BE.
- `SND_HDA_INTEL` y `SND_HDA_CODEC_REALTEK` para el audio.
- `USB_XHCI_PCI` y `USB_STORAGE` para los USB y pendrives.
- `EXT4_FS` para las particiones Linux.
- `VFAT_FS`, `EFI` y `EFI_STUB` para UEFI y la partición EFI.
- Bluetooth, cámara y los controladores que aparecían cargados en `lsmod`.

Comprobación final de los módulos críticos en `.config`:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ grep -E
'^CONFIG_BLK_DEV_NVME=|^CONFIG_NVME_CORE=|^CONFIG_VMD=|^CONFIG_USB_XHCI_PCI=|^CO
NFIG_DRM_I915=|^CONFIG_RTW89_8852BE=|^CONFIG_SND_HDA_CODEC_REALTEK=|^CONFIG_USB_
STORAGE=|^CONFIG_EXT4_FS=|^CONFIG_VFAT_FS=' .config
CONFIG_VMD=m
CONFIG_NVME_CORE=m
CONFIG_BLK_DEV_NVME=m
CONFIG_RTW89_8852BE=m
CONFIG_DRM_I915=m
CONFIG_SND_HDA_CODEC_REALTEK=m
CONFIG_USB_XHCI_PCI=m
CONFIG_USB_STORAGE=m
CONFIG_EXT4_FS=m
CONFIG_VFAT_FS=m
```

Guardé tanto la configuración final como el listado de módulos usado para hacer el recorte:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ cp .config ~/practica-
kernel/config-7.2.6-sergio-aso-final
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ cp ~/practica-
kernel/modulos-usados.txt ~/practica-kernel/modulos-usados-final.txt
```

---

## 6. Compilación y creación de paquetes

Para no duplicar el sufjo de versión configuré el kernel con `-sergio-aso` como versión local y generé los paquetes sin volver a pasar `LOCALVERSION`.

Primero compilé usando todos los hilos disponibles:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ make -j"$(nproc)"
```

Después generé los paquetes Debian:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ make -j"$(nproc)" bindeb-
pkg KDEB_PKGVERSION=7.2.6-sergio-aso-1
```

Los paquetes finales generados fueron:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ ls -lh ~/practica-
kernel/*.deb
-rw-r--r-- 1 sergio sergio  11M oct 12 20:08 /home/sergio/practica-kernel/linux-
headers-7.2.6-sergio-aso_7.2.6-sergio-aso-1_amd64.deb
-rw-r--r-- 1 sergio sergio  29M oct 12 20:08 /home/sergio/practica-kernel/linux-
image-7.2.6-sergio-aso_7.2.6-sergio-aso-1_amd64.deb
-rw-r--r-- 1 sergio sergio 228M oct 12 20:10 /home/sergio/practica-kernel/linux-
image-7.2.6-sergio-aso-dbg_7.2.6-sergio-aso-1_amd64.deb
-rw-r--r-- 1 sergio sergio 1,5M oct 12 20:07 /home/sergio/practica-kernel/linux-
libc-dev_7.2.6-sergio-aso-1_amd64.deb
```

Los dos ficheros importantes para instalar son el paquete de imagen y el de cabeceras:

```
linux-image-7.2.6-sergio-aso_7.2.6-sergio-aso-1_amd64.deb
linux-headers-7.2.6-sergio-aso_7.2.6-sergio-aso-1_amd64.deb
```

Comprobé la información del paquete de imagen:

```
sergio@Sergio-PC:~/practica-kernel/linux-source-7.2$ dpkg-deb -I ~/practica-
kernel/linux-image-7.2.6-sergio-aso_7.2.6-sergio-aso-1_amd64.deb | grep -E
'Package|Version|Architecture|Description'
 Package: linux-image-7.2.6-sergio-aso
 Version: 7.2.6-sergio-aso-1
 Architecture: amd64
 Description: Linux kernel, version 7.2.6-sergio-aso
```

---

## 7. Secure Boot e instalación

Al principio Secure Boot estaba activado. Como mi kernel era propio y no estaba firmado con una clave aceptada por la UEFI, elegí desactivar Secure Boot desde la configuración UEFI del portátil. Después lo comprobé desde Debian:

```
sergio@Sergio-PC:~$ mokutil --sb-state
SecureBoot disabled
```

Instalé solo los paquetes definitivos. Los kernels oficiales se mantuvieron instalados para poder volver atrás si algo salía mal:

```
sergio@Sergio-PC:~$ cd ~/practica-kernel
sergio@Sergio-PC:~/practica-kernel$ sudo apt install \
  ./linux-image-7.2.6-sergio-aso_7.2.6-sergio-aso-1_amd64.deb \
  ./linux-headers-7.2.6-sergio-aso_7.2.6-sergio-aso-1_amd64.deb
```

Durante la instalación se creó el initramfs y GRUB detectó el nuevo kernel:

```
Configurando linux-image-7.2.6-sergio-aso (7.2.6-sergio-aso-1) ...
update-initramfs: Generating /boot/initrd.img-7.2.6-sergio-aso
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-7.2.6-sergio-aso
Found initrd image: /boot/initrd.img-7.2.6-sergio-aso
Found linux image: /boot/vmlinuz-6.12.111+deb13-amd64
Found initrd image: /boot/initrd.img-6.12.111+deb13-amd64
Found linux image: /boot/vmlinuz-6.12.107+deb13-amd64
Found initrd image: /boot/initrd.img-6.12.107+deb13-amd64
```

---

## 8. Pruebas del kernel compilado

Tras reiniciar y elegir el kernel nuevo, comprobé que realmente estaba funcionando:

```
sergio@Sergio-PC:~$ uname -r
7.2.6-sergio-aso
```

La red funcionaba y la Wi-Fi estaba levantada con una dirección IP:

```
sergio@Sergio-PC:~$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
wlp1s0           UP             192.168.0.251/24 fe80::5231:23ff:feb4:2910/64
```

El almacenamiento también estaba correctamente reconocido:

```
sergio@Sergio-PC:~$ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
NAME                    SIZE TYPE FSTYPE      MOUNTPOINTS
nvme0n1               476,9G disk
├─nvme0n1p1             243M part vfat        /boot/efi
├─nvme0n1p2             954M part ext4        /boot
└─nvme0n1p3           475,8G part LVM2_member
  ├─vg_debian-lv_root  46,6G lvm  ext4        /
  ├─vg_debian-lv_home 265,4G lvm  ext4        /home
  ├─vg_debian-lv_var   96,6G lvm  ext4        /var
  ├─vg_debian-lv_tmp    7,4G lvm  ext4        /tmp
  └─vg_debian-lv_swap  14,9G lvm  swap        [SWAP]
```

Comprobé que se cargaban los módulos esenciales:

```
sergio@Sergio-PC:~$ lsmod | grep -E
'nvme|vmd|rtw89|i915|snd_hda|snd_sof|xhci|btusb|bluetooth|uvcvideo'
rtw89_8852be           12288  0
rtw89_8852b           344064  1 rtw89_8852be
rtw89_pci             114688  1 rtw89_8852be
rtw89_core            999424  3 rtw89_8852b,rtw89_pci,rtw89_8852b_common
btusb                  86016  0
bluetooth            1204224  34 btrtl,btmtk,btintel,btbcm,bnep,btusb,rfcomm
i915                 4784128  21
xhci_pci               24576  0
xhci_hcd              405504  1 xhci_pci
nvme                   69632  7
nvme_core             258048  8 nvme
vmd                    28672  0
```

También comprobé con `lspci -k` que los controladores en uso eran los correctos:

```
sergio@Sergio-PC:~$ lspci -k
00:02.0 VGA compatible controller: Intel Corporation Meteor Lake-P [Intel
Graphics] (rev 08)
    Kernel driver in use: i915
...
01:00.0 Network controller: Realtek Semiconductor Co., Ltd. RTL8852BE PCIe
802.11ax Wireless Network Controller
    Kernel driver in use: rtw89_8852be
...
02:00.0 Non-Volatile memory controller: Shenzhen Techwinsemi Technology Co.,
Ltd. PCIe Gen4 x4 M.2 2280 (rev 01)
    Kernel driver in use: nvme
```

Aparecieron algunos avisos ACPI, un aviso de firmware de Realtek y avisos DMAR en `dmesg`. Aun así, el sistema arrancó, el disco funcionaba, tenía conexión Wi-Fi, gráfica, audio, USB, cámara y Bluetooth. Por tanto, no afectaron al funcionamiento básico que se pedía en la práctica.

El único servicio que figuró como fallido fue `NetworkManager-wait-online.service`, pero la Wi-Fi estaba conectada y con IP, así que no impedía el uso de la red:

```
sergio@Sergio-PC:~$ systemctl --failed --no-pager
  UNIT                                LOAD   ACTIVE SUB    DESCRIPTION
● NetworkManager-wait-online.service loaded failed failed Network Manager Wait
Online
```

---

## 9. Arranque directo desde UEFI sin GRUB

Para cumplir el requisito de arrancar sin la mediación de un gestor de arranque, primero comprobé que el equipo estaba usando UEFI, que la partición EFI era `/dev/nvme0n1p1` y que tenía espacio suficiente.

```
sergio@Sergio-PC:~$ test -d /sys/firmware/efi && echo "Arranque UEFI: sí" ||
echo "Arranque UEFI: no"
Arranque UEFI: sí

sergio@Sergio-PC:~$ df -h /boot/efi
S.ficheros     Tamaño Usados  Disp Uso% Montado en
/dev/nvme0n1p1   240M   9,0M  231M   4% /boot/efi

sergio@Sergio-PC:~$ cat /proc/cmdline
BOOT_IMAGE=/vmlinuz-7.2.6-sergio-aso root=/dev/mapper/vg_debian-lv_root ro quiet
splash zswap.enabled=1 quiet
```

Copié el kernel y su initramfs a una carpeta propia dentro de la partición EFI:

```
sergio@Sergio-PC:~$ sudo mkdir -p /boot/efi/EFI/sergio-aso
sergio@Sergio-PC:~$ sudo cp /boot/vmlinuz-7.2.6-sergio-aso \
  /boot/efi/EFI/sergio-aso/vmlinuz-7.2.6-sergio-aso.efi
sergio@Sergio-PC:~$ sudo cp /boot/initrd.img-7.2.6-sergio-aso \
  /boot/efi/EFI/sergio-aso/initrd-7.2.6-sergio-aso.img
sergio@Sergio-PC:~$ sudo ls -lh /boot/efi/EFI/sergio-aso
total 56M
-rwx------ 1 root root 44M oct 12 20:37 initrd-7.2.6-sergio-aso.img
-rwx------ 1 root root 13M oct 12 20:36 vmlinuz-7.2.6-sergio-aso.efi
```

Después creé una entrada UEFI directa con `efibootmgr`. La entrada apunta al kernel EFI stub, le pasa el initramfs y le indica la raíz LVM:

```
sergio@Sergio-PC:~$ sudo efibootmgr --create \
  --disk /dev/nvme0n1 \
  --part 1 \
  --label "Sergio ASO - Kernel directo 7.2.6" \
  --loader '\EFI\sergio-aso\vmlinuz-7.2.6-sergio-aso.efi' \
  --unicode 'root=/dev/mapper/vg_debian-lv_root ro quiet splash zswap.enabled=1
initrd=\EFI\sergio-aso\initrd-7.2.6-sergio-aso.img'
```

La entrada creada fue `Boot0001`:

```
Boot0001* Sergio ASO - Kernel directo 7.2.6 HD(1,GPT,9b35a92e-3d9b-4887-a680-
d76e0a0eaf2b,0x800,0x79800)/File(\EFI\sergio-aso\vmlinuz-7.2.6-sergio-
aso.efi)root=/dev/mapper/vg_debian-lv_root ro quiet splash zswap.enabled=1
initrd=\EFI\sergio-aso\initrd-7.2.6-sergio-aso.img
```

Para no dejar el arranque normal modificado, dejé GRUB primero en el orden permanente y probé la entrada directa una sola vez mediante `BootNext`:

```
sergio@Sergio-PC:~$ sudo efibootmgr -o 0003,0001,0000
sergio@Sergio-PC:~$ sudo efibootmgr -n 0001
sergio@Sergio-PC:~$ sudo efibootmgr
BootNext: 0001
BootCurrent: 0003
BootOrder: 0003,0001,0000
```

Después del reinicio, la prueba fue correcta. El kernel arrancó directamente desde UEFI y no aparecía `BOOT_IMAGE=`, que es el parámetro que añadía GRUB:

```
sergio@Sergio-PC:~$ uname -r
7.2.6-sergio-aso
sergio@Sergio-PC:~$ cat /proc/cmdline
root=/dev/mapper/vg_debian-lv_root ro quiet splash zswap.enabled=1
initrd=\EFI\sergio-aso\initrd-7.2.6-sergio-aso.img
```

Con esto queda comprobado que el arranque fue sin pasar por GRUB.

## 10. Conclusión

He compilado el kernel `7.2.6-sergio-aso` desde el código fuente de `trixie-backports`, usando como base la configuración de mi kernel Debian original.

Después reduje la configuración con `localmodconfig` a partir de los módulos cargados en mi equipo y revisé manualmente los drivers necesarios para no perder soporte de disco, gráfica, Wi-Fi, audio, USB, cámara, Bluetooth, EFI y sistemas de archivos.

Generé e instalé los paquetes `.deb`, comprobé que el kernel arrancaba y que el hardware básico funcionaba. Por último, preparé y probé una entrada UEFI que arranca directamente el kernel con EFI stub, sin utilizar GRUB.

Como medida de seguridad, mantuve los kernels oficiales y la entrada Debian/GRUB disponibles como recuperación.